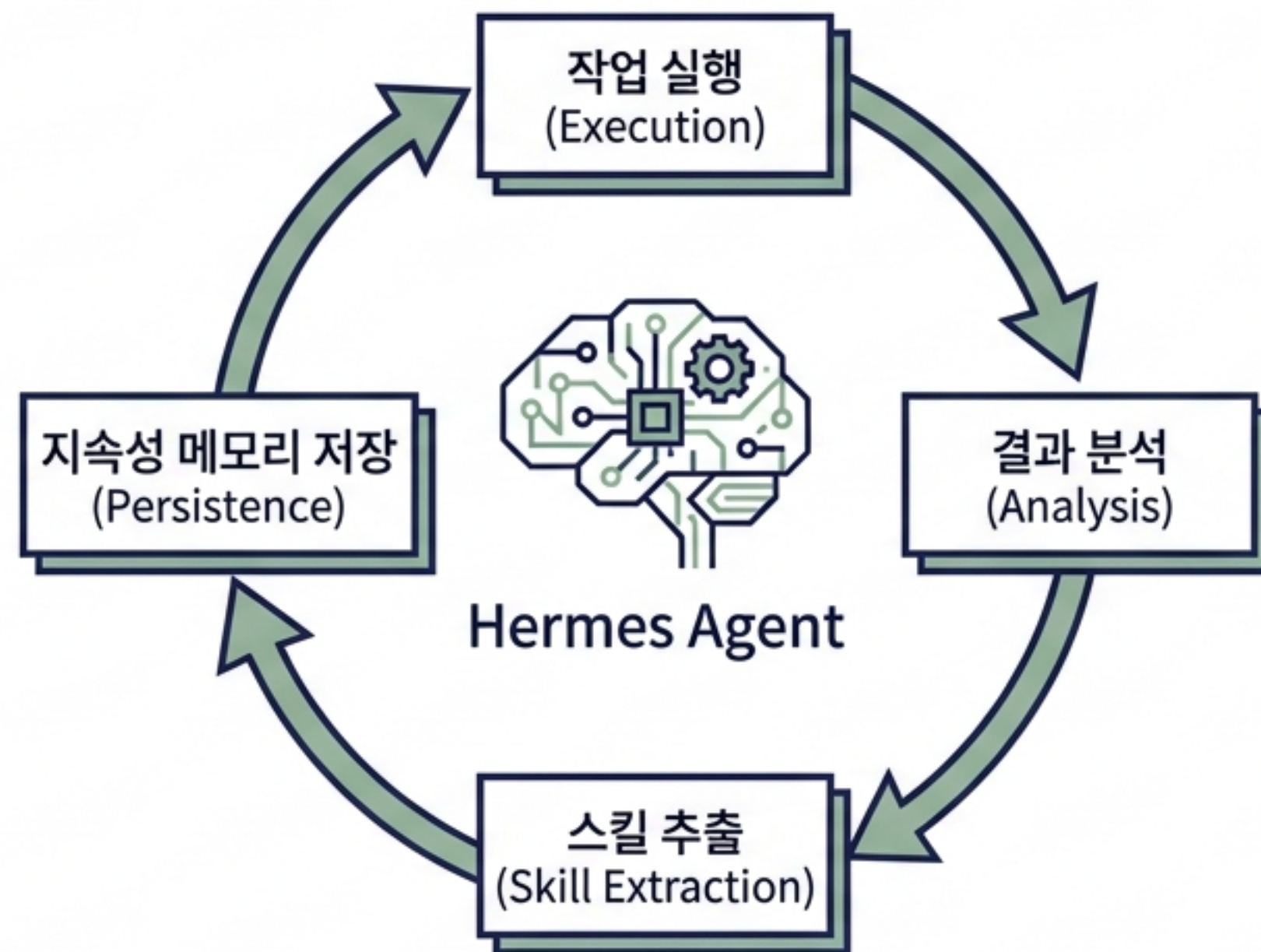


5교시: 스스로 진화하는 AI, Hermes Agent의 핵심 가치



경험 기반 성장

일회성 답변을 넘어,
작업을 기억하고 내재화하는 시스템

자기 개선 루프

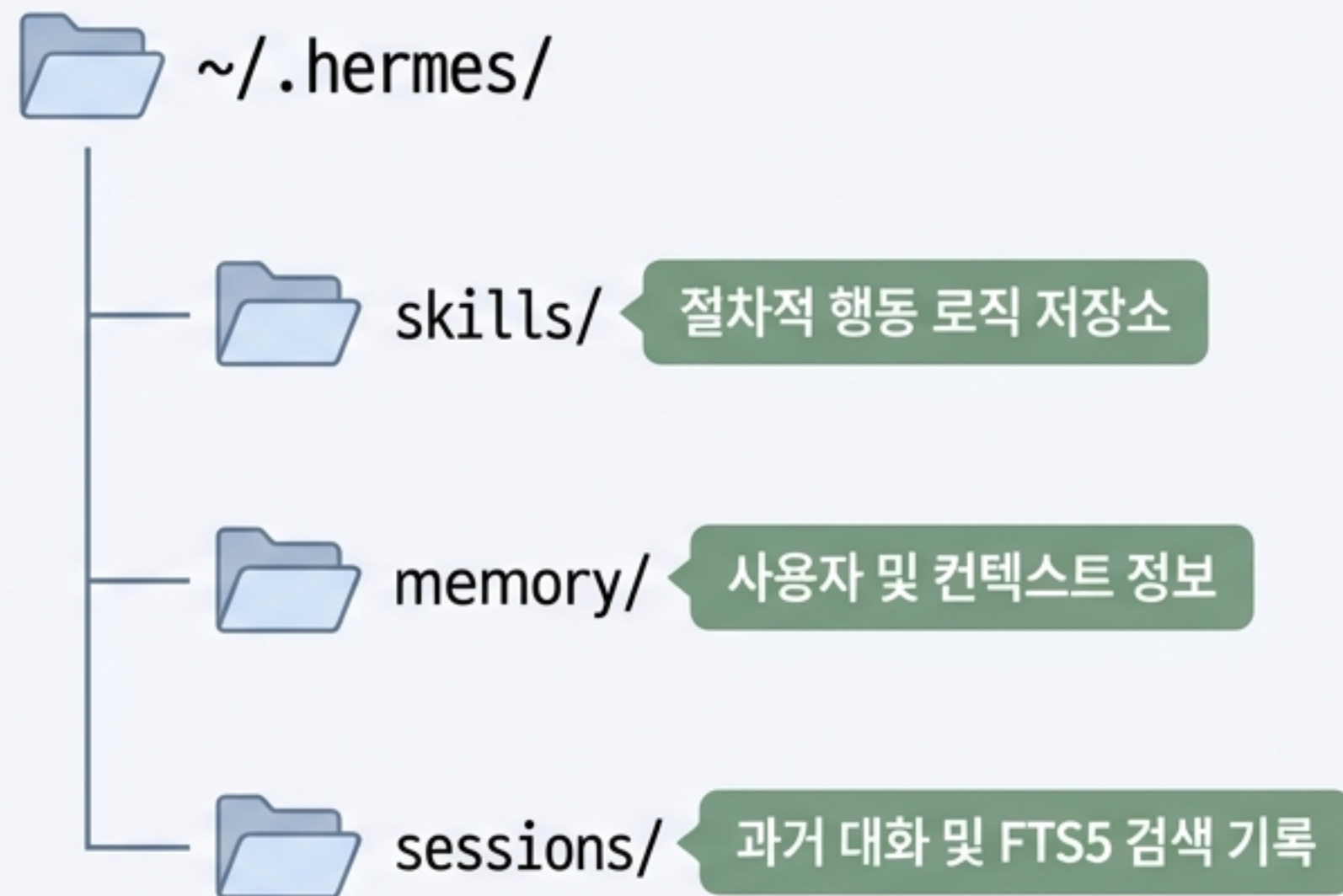
실행과 피드백을 통해
스스로 다음 행동을 최적화

맞춤형 자산화

저비용 인프라에서도 동작하는
기업 고유의 디지털 두뇌

환경 세팅 및 아키텍처: 에이전트의 두뇌 구조

```
1 설치 명령어 실행:  
2 $ curl -sL https://hermes-agent.io/install | bash  
3  
4 ✓ 설치 환경: Linux, macOS, WSL2 완벽 지원  
5  
6 ✓ 멀티 모델 유연성: 'hermes model' 명령어로  
7 락인(Lock-in) 없이 즉각 교체  
8 |
```



코드 수정 없이 인프라와 모델을 분리하여 통제하는 완벽한 유연성

TUI(터미널 UI) 구조의 완벽 해부

대화 히스토리 창

멀티라인 편집 및
실시간 스트리밍 지원

컨텍스트 모니터

현재 활성화된 모델 및
토큰 상태 실시간 추적

명령어 입력창

Ctrl+C: 작업 즉각 중단 (Interrupt)

방향 재설정 및 개입 (Redirect) 메커니즘

슬래시(/) 커맨드를 활용한 에이전트 완벽 제어

커맨드	핵심 기능	제어 목적
/new, /reset	컨텍스트 및 세션 초기화	새로운 작업 환경 구성
/model, /personality	에이전트 지능 엔진 및 성격 변경	작업 성격에 맞춘 뇌 구조 교체
/skills	활성화된 능력 및 도구 리스트 확인	가용 리소스 점검
/compress	대화 히스토리 토큰 압축	컨텍스트 윈도우 최적화

```
> /  
  /model  
  /new  
  /personality  
  /skills
```

복잡한 설정 파일 없이 단축키 하나로 통제실 제어 완료

[실습] 첫 번째 작업 지시와 자율 도구 활용



단순한 응답을 넘어, 에이전트가 스스로 도구를 선택하고 파일 시스템에 접근하는 자율성 검증

6교시: 지속성(Persistence) 원리 - AI의 기억 시스템

선언적 기억 (Declarative Memory)



- 단순 사실과 데이터의 파편적 저장
- 기존의 지식 기반 RAG 방식
- 맥락이 단절될 위험 존재

절차적 기억 (Procedural Memory)

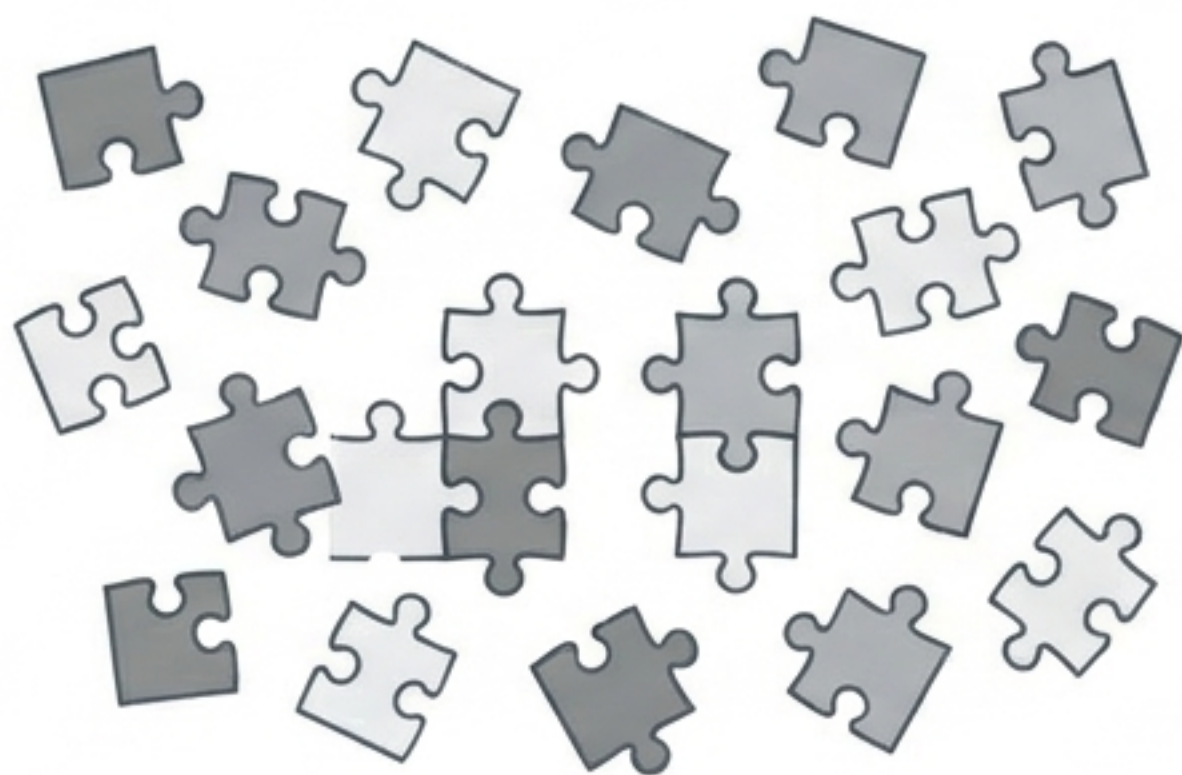


- 문제 해결 '방법론'과 '순서'의 저장
- Skill Document를 통한 행동 내재화
- 컨텍스트가 초기화되어도 능력 영구 유지

Hermes의 차별점: 주기적으로 자신의 경험을 정리하고 스스로를 넛지(Nudge)하여 기억을 영구 보존

한계 돌파: 단순 RAG vs 구조화된 마크다운

Vector RAG (기존 방식)



- 🔗 • 문맥 단절 및 절차적 뉘앙스 상실
- ? • 불필요한 노이즈 검색으로 인한 환각(Hallucination) 발생
- 🧩 • 데이터의 파편적 결합

Structured Markdown (Skill Document)



- 📄 • clawxiv 논문에서 입증된 최적의 전송 포맷
- ⚙️ • 하나의 문서에 전제조건, 실행 단계, 검증 방법 통합
- ✅ • LLM이 가장 완벽하게 파싱할 수 있는 네이티브 포맷

Skill Document 해부학: AI의 완벽한 행동 지침서

메타데이터 (YAML):
에이전트가 문서를
로드할지 판단

```
1  ---
2  name: 데이터_정합성_검증
3  description: 주간 리포트 데이터 검증 및 병합
4  ---
5
6  ## Trigger
7  사용자가 '주간 데이터 확인해줘'라고 요청할 때 발동
8
9  ## Steps
10 1. DB_A와 DB_B에서 최근 7일 데이터를 추출한다.
11 2. 두 데이터의 고유 ID를 기준으로 병합한다.
12
13 ## Troubleshooting
14 데이터 타입 오류 발생 시, string으로 강제 형변환 후 재시도한다.
15
```

발동 조건: 언제
이 스킬을 꺼내 쓸
것인가 명시

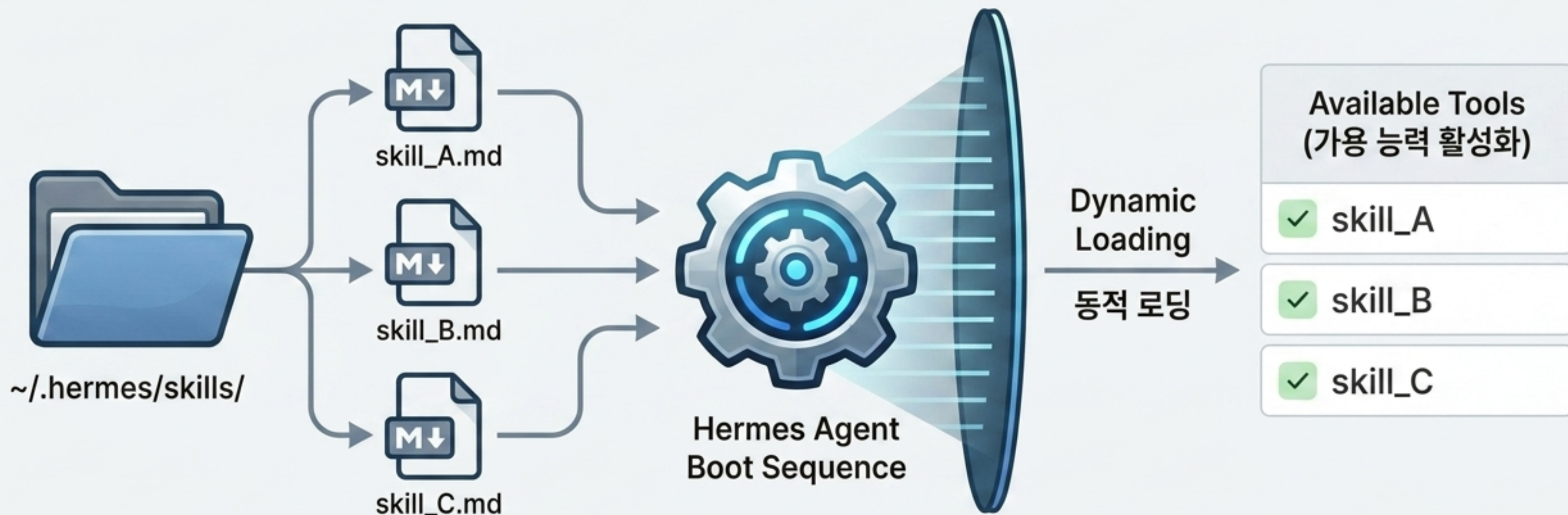
실행 절차:
명확한 Action 기반의
순서도

예외 처리 규칙:
실패 시 대응 방안을
명시하여 무한 루프 방지

[실습 설계] 우리 부서의 첫 번째 Skill Document 기획

1. 실무 병목 정의 (Target Task) 회사에서 반복적으로 발생하는 데이터 취합, 정합성 검증 등의 병목 업무 선정 예: 매주 월요일 주간 마케팅 지표 취합	2. 필요 도구 (Required Tools) 업무 수행에 필요한 시스템 권한 및 외부 도구 예: 파일 시스템 읽기/쓰기, Python 실행
3. 절차적 분해 (Deconstruction) AI가 수행할 수 있는 3~4단계의 명확한 순서도 예: Step 1. 엑셀 로드 -> Step 2. 빈 값 제거 -> Step 3. 요약	4. 가변/고정 데이터 분리 (Success Criteria) 매번 바뀌는 변수와 고정된 룰을 분리하여 성공 조건 명시 예: 완성된 요약본이 마크다운 표 형태로 출력될 것

기억의 동기화: 지식 기반에서 행동 기반으로의 전환



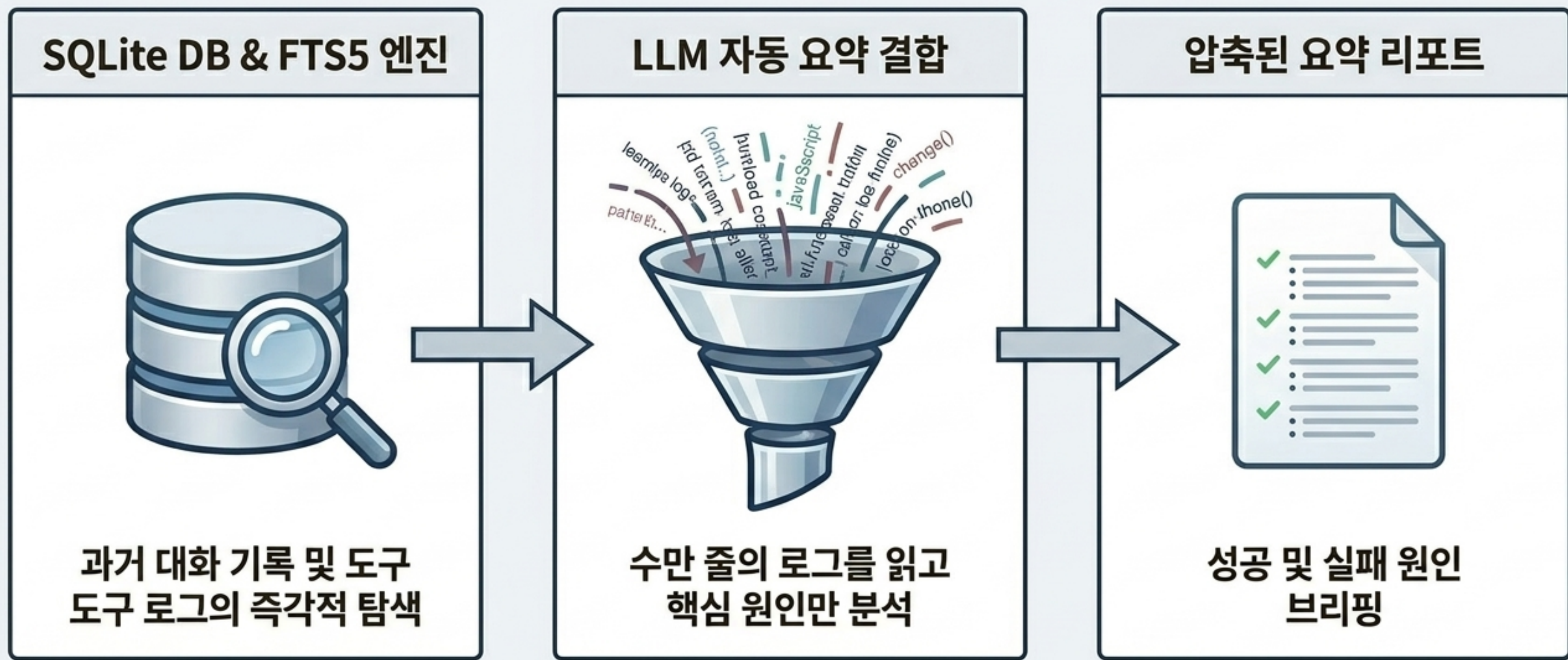
로컬 스토리지 매핑

기획한 스킬이 물리적 마크다운 파일로 저장됨

수동 편집 및 즉각 교정

실무자가 문서를 직접 수정하여 AI의 로직을 실시간으로 통제 가능

7교시: 과거 경험의 회상 (FTS5 세션 검색)



[실무 실습] 복잡한 과제 해결과 에러 극복

복잡한 과제 지시서



시나리오:

다중 데이터 소스(3개의 엑셀 파일)를 병합하고 리포트 포맷팅을 수행하라.

목적:

의도적으로 발생하는 파싱 에러를 에이전트가 스스로 극복하는 과정 관찰.

터미널 모니터링 (Human-in-the-Loop)

❗ [Error] 파일 포맷 불일치로 병합 실패

🔄 [Reasoning] 파이썬 스크립트 수정 후 재시도 필요

/redirect 명령어를 통한 방향 가이드 수신

✅ [Success] 데이터 정합성 검증 및 리포트 도출 완료

경험의 내재화: 새로운 '스킬(Skill)' 자동 추출

시행착오 터미널 로그
(Raw Experience)

```
Error: File not found /usr/bin/...  
TRACEBACK: [Main Process]  
  File "/envon/devages.py", line 217,  
    return sport (file_name )  
[WARNING] Connection timeout ...  
[WARNING] Connection timeout ...  
...  
raw_data = load_csv('data.csv')  
print(f'Processing {file_name}')  
file_name = load_data  
print(f'Processing {file_name}')  
...  
raw_data =
```

AI Filter & 구조화

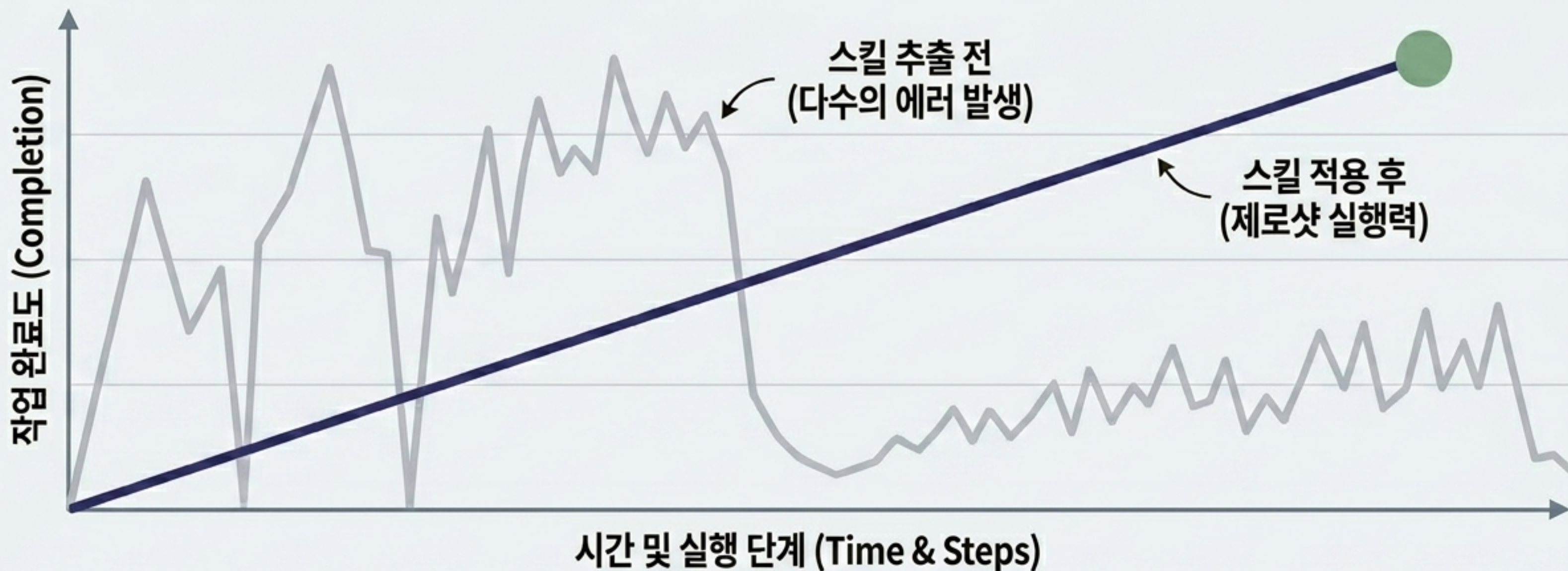


Skill_Data_Merge.md



- 경험 추출 지시: 자연어 명령 한 번으로 방금의 성공 궤적 분석
- 메타데이터 자동 생성: 스킬 발동 조건(Trigger) 스스로 정의
- 결과물 검수: 불필요한 단계를 제거하고 구조화된 마크다운으로 압축

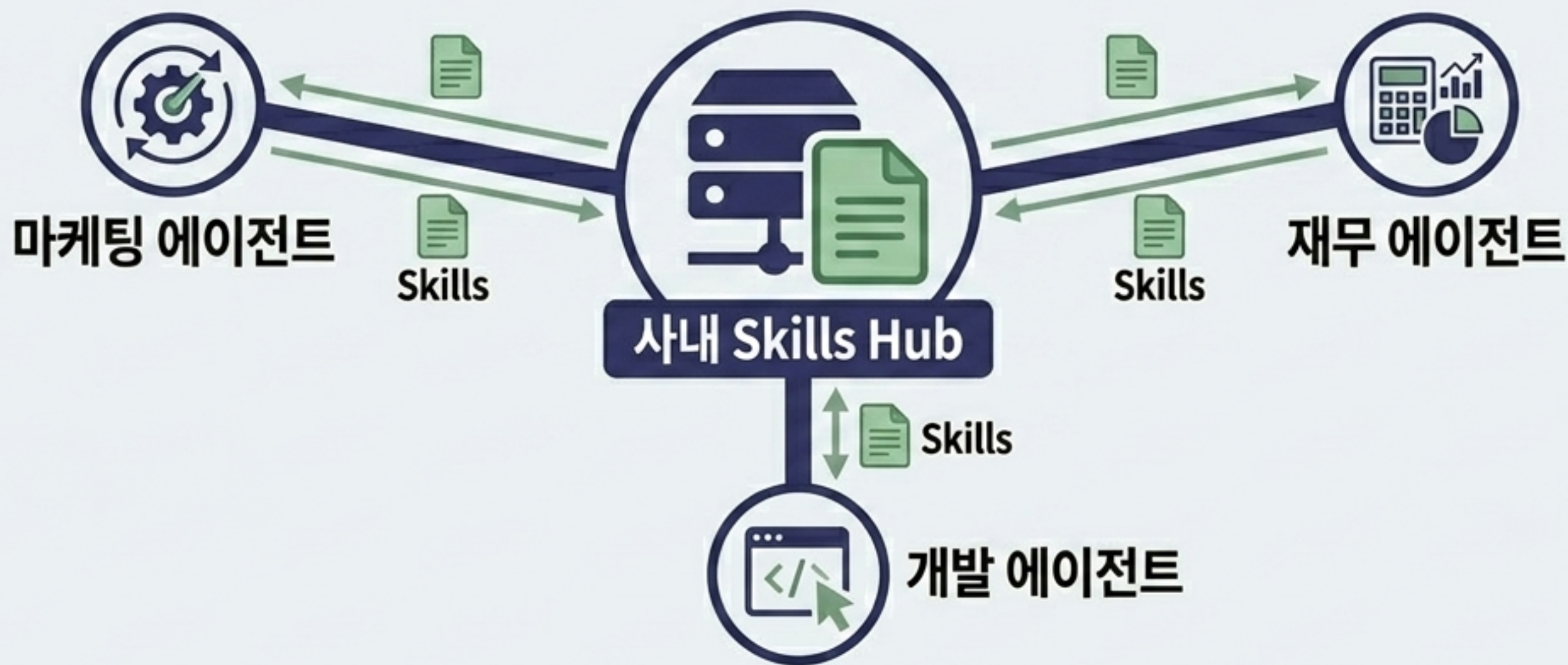
추출된 스킬의 즉각적 적용 및 성능 입증



- 완전히 새로운 데이터셋에도 동일한 스킬 적용 가능
- 절차적 기억을 통한 에러 확률 급감

- 작업 처리 속도의 비약적 상승
- 사소한 오류는 실무자가 마크다운을 직접 편집해 즉시 디버깅

자기 개선 루프의 실무 적용 Best Practice

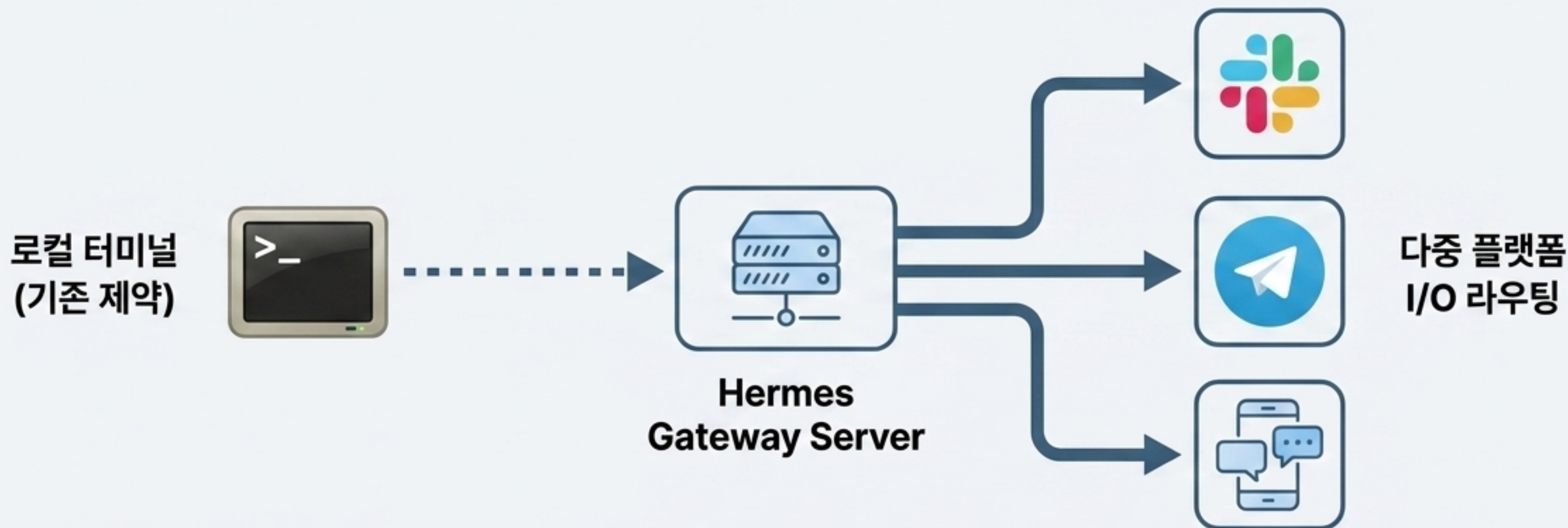


지속 가능한 진화
반복 빈도가 높고 구조화
가능한 업무만 스킬로 타겟팅

Human-in-the-Loop
시스템 영구 저장 전,
인간 디렉터의 로직 검수 필수

조직적 지능 향상
개인이 습득한 마크다운 문서를
사내 전체 에이전트와 공유

8교시: 에이전트 게이트웨이 - 터미널을 넘어 메신저로



제약 극복

로컬 PC나 터미널 화면에 종속된
인터페이스 한계 탈피

클라우드 상시 대기

모바일에서도 언제든지 지시하고
실시간으로 보고받는 환경

대화 연속성

플랫폼 고유의 특성(음성 메모 등)을
활용한 유비쿼터스 연동

멀티 플랫폼(Telegram/Slack) 연동 실습



1. 플랫폼 봇 생성

BotFather 또는
App Directory를
통한 인증 키 발급



2. 환경 변수 구성

.env 파일에
API 토큰 및 사용자
접근 권한 설정



3. 게이트웨이 실행

'hermes
gateway start'
백그라운드 리스너
구동



4. 스마트폰 연동

메신저 앱으로
지시 전달 및
결과물(파일/텍스트)
수신

터미널 검은 화면을 벗어나 전 세계 어디서든
에이전트에게 명령을 내릴 준비 완료

무인 자동화의 완성: 자연어 기반 크론(Cron) 스케줄러

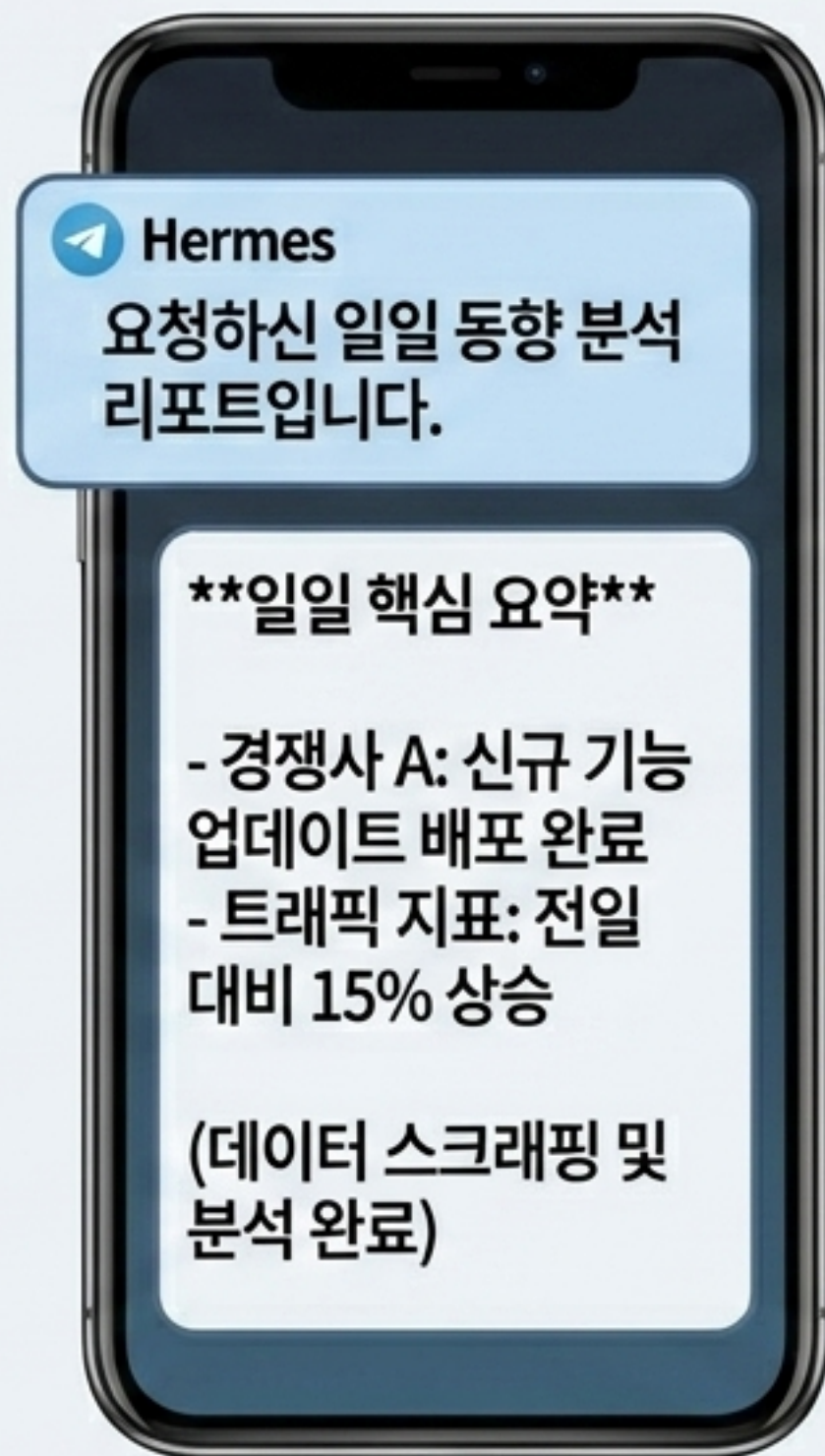


전통적 Cron의 한계:
복잡한 별표 문법(* * * * *)과
고정 스크립트의 경직성

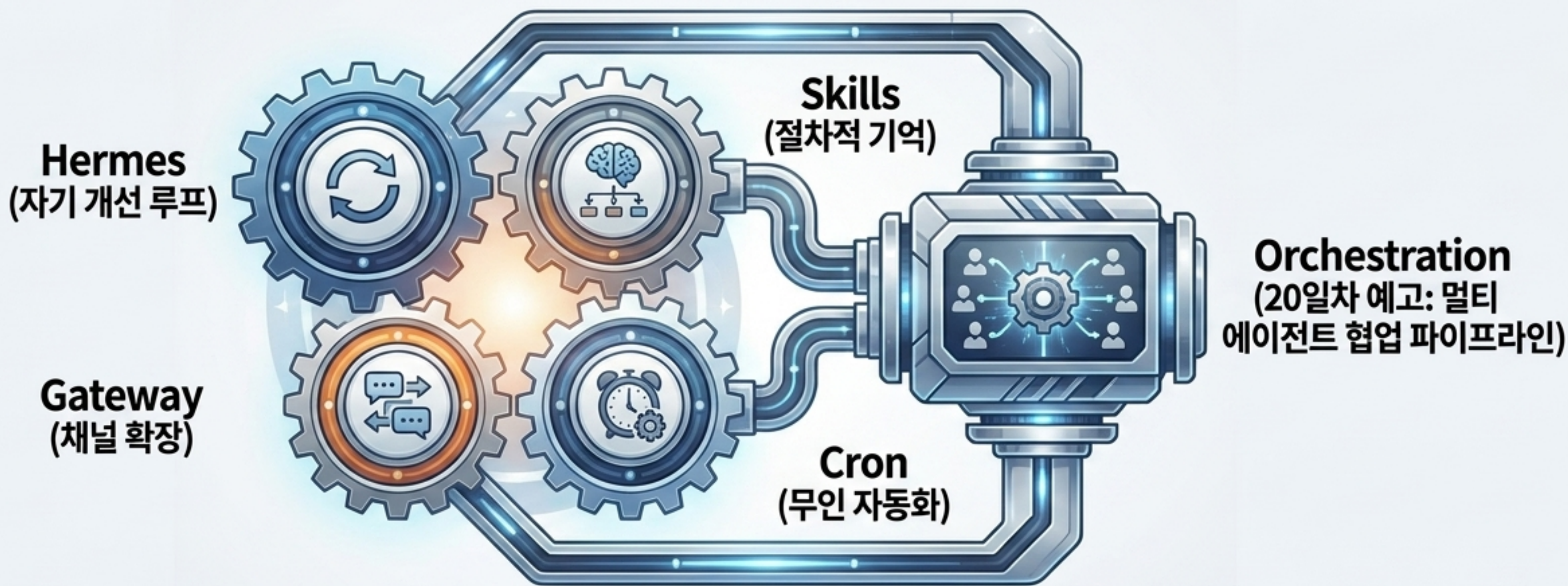
자연어 스케줄러:
개발 지식 없이 일상 언어만으로
진정한 무인 백그라운드 자동화 구현

[실습] 백그라운드 자동화 파이프라인 구축 (일일 보고서)

- 실습 미션: 웹사이트 데이터 스크래핑 및 요약본 메신저 정기 보고
- 스케줄링 등록: 메신저에서 자연어로 정기 작업 지시
- 수동 트리거(Trigger) 테스트:
'context_from' 기능을 통해 크론 잡 강제 실행 및 퀄리티 선제 검수



Wrap-up: 자율 조직(Autonomous Org)의 디렉터로 도약



단순한 AI 도구 사용자를 넘어, 시스템이 스스로 일하게 만드는 ‘설계자(Architect)’로의 진화.
AX 시대, 대체 불가능한 ‘인간 디렉터(Human Director)’의 가치 창출.